

Swarmbly AI

The barrier to serving artificial intelligence stops being capital and becomes participation

Sebastián Espinoza · Pontificia Universidad Católica del Ecuador · University of Saskatchewan Whitepaper v1.4 · specification v0.2 · reference implementation and first measurements published AGPL-3.0-or-later (software) · CC BY 4.0 (text) · github.com/Sebastardito/Swarmbly-AI

The asymmetry

The knowledge to build artificial intelligence is public. Model weights, training recipes and inference engines are published openly and improve every month. **The capital to operate it is not.** Data centres consumed 415 TWh in 2024 — about 1.5 % of world electricity — with projections to 945 TWh by 2030, and that growth path runs through construction, which is available only to whoever can finance it.

So a technology whose knowledge belongs to everyone ends up controlled by whoever can afford the buildings. Not through a patent. Through a power contract.

Meanwhile the hardware already exists, switched on and idle. The flagship volunteer-computing platform today aggregates roughly **700,000 active devices, 4 million CPU cores, 560,000 GPUs and 93 PetaFLOPS** — from a community that has *shrunk* by 80 % over two decades. That number is a floor, drawn from a single declining niche, not a projection. At the level of an individual machine, an idle RTX 4090 is reported to serve language-model inference at **\$0.111-0.149 per million tokens**, at 62-78 % of H100 throughput for roughly half the cost.

The world's spare inference capacity is not a hypothesis. What has been missing is a protocol under which it can be used.

Why nobody has managed it yet

Every serious attempt so far has split the **model** — distributing transformer layers across machines so that intermediate activations cross the public internet on every generated token. That design runs head-first into a wall of physics: datacenter interconnect moves 900 GB/s, consumer upstream moves about 60 Mbps. **A ratio of roughly 120,000×**, and four to five orders of magnitude on latency.

The measured results match the prediction. Petals, the reference implementation of this approach, drops 31 % of its throughput to the network alone when moved from a lab link to a realistic one; a real geodistributed swarm of fourteen servers manages 0.83 steps per second.

That is not a bad implementation. It is the right answer to the wrong question.

The reframe

Swarmbly asks a different question: not *how do you run one large model across many machines*, but **how do you run many complete small models on one large problem**.

A small orchestrator on the user's own computer decomposes a request into semantic micro-tasks. Each is dispatched **once**, asynchronously, to a volunteer node running a complete small model. The returned fragments — *contigs*, in the genome-assembly vocabulary the design borrows deliberately — are verified, selected and spliced back together locally.

The network is crossed once per fragment per session, instead of once per layer per token. That single change moves the architecture from the side of the 120,000× wall where it loses, to the side where consumer hardware can participate at all. Splitting a model creates a chain, where every machine waits on the one before it. Splitting a problem creates a set, where they all work at once. That contrast is architectural: Swarmbly's own throughput and latency have not been measured yet, and this page makes no speed claim on their behalf.

What has already been measured

The design rests on one falsifiable claim: **the more shared context each fragment carries, the less quality is lost when the pieces are rejoined**. A go/no-go threshold was registered publicly *before any data existed* — if the loss never fell below 5 % in any task category, the architecture was to be abandoned.

It has now been run against three real model families. The prediction held:

Shared context (p)	Quality lost to fragmentation
1.00	24.1 %
1.25	20.4 %
1.50	16.1 %
2.00	13.7 %

Monotone, in both the ratio and the denominator-free absolute difference. **Three task categories cleared the pre-registered threshold, and in two of them the tax went negative — fragmenting the problem and reassembling it produced a *better* answer than doing it in one piece, by as much as 9.0 %.**

That is the core argument, measured rather than asserted: the variable the design says is the control variable behaves like the control variable. Two caveats travel with that table rather than being dropped from it — the figures come from one of two coherence instruments, the second having produced no usable measurement on this corpus, and one of the eight prompts failed to produce a baseline at all, so its cells are excluded.

What has not been proven — stated here, not buried

One published contribution did **not** survive its first test. The architecture returns a *confidence map* — because independent model families answer the same micro-task, their agreement can be scored per unit, and a single centralized provider has nothing to align. The mechanism works. But the first measurement found **no relationship between agreement and judged quality** ($r = -0.030$ over 597 units). The instrument was weak — the automated judge accepted 93 % of everything — so the honest statement is that the property is **unsupported, not refuted**. It has been demoted in the whitepaper accordingly, in the same document that first advertised it.

The measurements are also small: eight prompts — one of them without a usable baseline — one seed, 2-3B models. A signal to act on, not a benchmark.

Four things this project does not claim. It is **not faster than a commercial API** for someone who already owns the hardware to run one — single-node speculative decoding beats any fragmentation scheme on latency, and Swarmby's own latency has not been measured. It does **not offer unlimited context**, only a much higher limit that sits on the user's machine instead of in a vendor's price tier. It is **not encryption**: fragmentation raises the cost of reconstruction and nothing more, which is why genuinely sensitive work is routed to a closed circle or kept entirely local. And it has **not demonstrated an environmental benefit** — the argument is strong, the measurement is not yet made, and the project commits to publishing it whatever it shows.

This section exists because a project that hides its first negative result has not earned the first positive one.

Why now, and what exists today

Small models crossed the capability line that makes this possible only recently; the bandwidth gap that killed model-splitting is not closing. The opportunity is a timing one.

Published and public as of August 2026: a 27-page whitepaper with 90 references, a complete wire specification, a reference implementation with **178 passing tests**, a labelled evaluation corpus, an experimental harness that reports its own measurement failures, and the first real measurements in full. Everything is AGPL-3.0-or-later so that a hosted deployment cannot close it, with a public-domain-dated prior-art record.

What it needs next is not funding first — it is participants. The likeliest way this fails is not an engineering fault; it is that nobody connects. Volunteer computing has been declining for twenty years, and the best protocol in the world is worth nothing to an empty network.

The knowledge is already public. The hardware is already built. What remains is the protocol, and it is now on the table where anyone can check it.

Full technical argument: docs/WHITEPAPER_EN.md. Plain-language version: docs/DIVULGACION_EN.md. Measurements in full: docs/RESULTS_V0_V3C.md. Spanish version of this page: ONEPAGER_ES.md.